

Secure Dockerfile Refactoring Tool (Go) – Research, Plan, and Implementation

Research

Overview of the Problem and Solution Domain

Container base images often include unnecessary packages and known vulnerabilities, increasing the security risk of applications. Many organizations start with popular bases like Debian or Alpine, only to find dozens of CVEs before adding any app code ¹. Slimmer alternatives like **Distroless** (by Google) and **Chainguard**'s Wolfi-based images offer minimal, secure-by-default bases that dramatically reduce vulnerability counts. For example, Chainguard advertises *zero known CVEs at release* for their images ² and reports a **97.6% reduction in CVEs compared to typical OSS base images** ³. In practice, **Debian-slim** images still carry many vulnerabilities, and even Google's Distroless has lagged in patching at times ⁴. Replacing Dockerfiles' base images with these secure alternatives can virtually eliminate known OS-level vulnerabilities, easing compliance and security scans ⁵ ⁶.

This research focuses on a Go-based tool to **scan Dockerfiles for vulnerabilities using Trivy**, then **rewrite the Dockerfiles to use secure base images (Chainguard or Distroless)**, and finally **log the results and generate before/after security reports**. Optionally, the tool will integrate with GitHub to automate creating a Pull Request (PR) with the proposed changes. The goal is to streamline the process of upgrading to safer base images and quantify the security improvement (e.g. fewer vulnerabilities) via SBOM and Trivy scan diffs.

Architecture and Key Components

The tool's architecture will be modular, leveraging existing libraries and CLIs where possible:

- **Dockerfile Parsing Module:** To analyze and rewrite Dockerfiles, the tool will parse the file to identify `FROM` instructions (including multi-stage builds). We can use Docker's official parser (from Moby/BuildKit) to get an AST of the Dockerfile ⁷. This allows handling complex Dockerfiles robustly (comments, multiple stages, etc.). The parser will find base image references (e.g. `FROM python:3.11-slim` as base stage, possibly builder stages too) and extract the image name and tag. For multi-stage Dockerfiles, typically only the final runtime stage's base is critical to secure (build stages can use larger images). The module will support rewriting the base image tag or name in the AST and outputting an updated Dockerfile.
- **Base Image Recommendation Logic:** A core decision point is selecting a secure replacement for a given base image. In straightforward cases (common languages or OS), the tool can map known images to their Chainguard or Distroless equivalents:

- For language runtime images (Node, Python, Java, etc.), use Chainguard images (e.g. `cgr.dev/chainguard/python:latest` for `python:3.X`) or official Distroless (`gcr.io/distroless/python3`).
- For generic Linux bases (Debian/Ubuntu/Alpine), use Wolfi-based minimal images (`cgr.dev/chainguard/wolfi-base` or `.../static:latest` for a scratch-like image).
- These images are minimalist and **often drop shells, package managers, and other non-essential components** ⁸ ⁹. The tool's documentation will note that **going distroless may require multi-stage builds and ENTRYPOINT adjustments** ¹⁰. In cases where a Dockerfile isn't already multi-stage and tries to install packages in a single stage, the tool might either warn the user or automatically insert a builder stage (future enhancement).
- An extension could be a **mapping file** that pairs common images with secure alternatives (maintained by the tool or user-configurable). Initially, we might hard-code a few popular mappings for demonstration.
- **Trivy Scanner Integration:** Trivy by Aqua Security is a powerful, open-source vulnerability scanner that can scan container images and file systems, and even generate SBOMs ¹¹ ¹². The tool will use Trivy in two ways:
 - **Vulnerability Scanning:** Before and after replacing the base image, we will run Trivy to scan for known vulnerabilities in the image. We have two options:
 - **Scan the base image alone:** Pull the original base image (e.g. `node:18-alpine`) and run `trivy image` on it, then do the same for the secure image (`cgr.dev/chainguard/node:latest`). This gives a direct comparison of OS package vulnerabilities in the base layers.
 - **Scan the full application image:** If possible, build the Dockerfile before and after the change and scan the whole image. This requires a Docker environment and could be time-consuming, so initially focusing on base image comparison might be simpler. It still highlights the majority of vulnerabilities which come from OS packages ¹³.
 - Trivy outputs include counts of vulnerabilities by severity (Critical/High/Medium/etc.) which we can log to show improvement. We might run Trivy with `--severity HIGH,CRITICAL` to focus on serious issues, or let it list all.
 - **SBOM Generation:** Trivy can **generate SBOMs in SPDX or CycloneDX format** as part of scanning ¹⁴. Using the CLI `--format` flag, we can produce a CycloneDX JSON SBOM for an image, e.g. `trivy image --format cyclonedx -o sbom_original.json <image>` ¹⁵. The tool will generate SBOMs for the original image and the new image. An SBOM (Software Bill of Materials) is essentially an inventory of all packages and components in the image. By diffing the "before" and "after" SBOM (or just comparing their contents manually), users can see exactly which components were removed or added by switching the base. For example, an Ubuntu-based image might list dozens of library packages while the Chainguard-based one lists only a handful (just what the app needs).
 - Using CycloneDX is recommended for broad compatibility ¹⁶. The tool does not need to implement SBOM diff logic itself; it can simply save the files and perhaps call Trivy's own SBOM scan to enumerate vulnerabilities from each SBOM ¹⁷ ¹⁸.

- The generated SBOMs also serve as an artifact for compliance, demonstrating what's in the images.
- **Logging and Reporting:** The tool will provide detailed logs and a final summary:
 - Logging (using a structured logging library like Logrus or Zap in Go) will record steps: which files are being processed, what base image is found, what replacement is chosen, etc. It will log vulnerability scan results (e.g. "Original base `ubuntu:22.04` had 50 known vulns (5 critical, 10 high) ⁴ ; new base `cgr.dev/chainguard/wolfi-base` has 0 known vulns ² "). Where possible, it should include key data from Trivy's output to justify the change.
 - The tool will produce or print a **before/after comparison**. For example:
 - **Base Image Change:** `FROM ubuntu:22.04` → `FROM cgr.dev/chainguard/wolfi-base:latest`
 - **Vulnerabilities:** Ubuntu base: 50 (5 Critical, 10 High); Wolfi base: 0 (0 Critical, 0 High).
 - **Image Size:** (if easily obtained, e.g. via Docker Hub API or after building, though this might be out of scope for MVP).
 - Possibly list the top critical CVEs that were eliminated.
 - The SBOM files (e.g. `original.sbom.json` and `new.sbom.json`) and Trivy scan reports (in JSON or table form) can be saved in an output directory for further analysis. The log might mention their location.
- **GitHub PR Integration (Optional):** To close the loop, the tool can automatically create a Pull Request with the Dockerfile changes:
 - This requires a GitHub token with repo access. Using the GitHub API (or a Go library like `google/go-github`), the tool can create a new branch in the repository, commit the modified Dockerfile(s), and open a PR. The PR's title and body will summarize the changes, e.g. "Secure Base Image: Replace `ubuntu:22.04` with Wolfi (zero-CVE base)", and detail the vulnerability reduction: "This change drops known vulnerabilities from 50 to 0 by using a Chainguard Wolfi base ² ⁶ . See attached SBOM and Trivy reports for details."
 - Logging will record whether the PR was created successfully or if any error occurred (e.g. permissions issues).
 - This feature can be toggled (e.g. `--github-pr` flag) so users without GitHub access can just do local changes. If enabled, the tool likely needs info like the Git repo URL/path, branch name (it could default to something like `secure-base-image` branch), and the GitHub token (likely via environment variable or a config file for security). It should handle forks if the user doesn't have direct push access (similar to how some bots fork then PR). Initially, assume direct access for simplicity.

Challenges and Considerations

- **Maintaining Image Replacements:** Not all base images have a one-to-one secure counterpart. The tool's effectiveness lies partly in maintaining an updated mapping of secure base images. Chainguard's catalog has 1,300+ images (as of 2025) ¹⁹ , covering many languages and services, but some specialized images might not have a drop-in replacement. We may restrict scope to common bases in the first iteration. The tool might print "No secure alternative found for X, skipping." in such cases.

- **Compatibility:** Simply switching the base image could break the Dockerfile or application:
- If the original base provided a shell or package manager that subsequent `RUN` commands use, switching to a distroless image (which has **no shell in runtime** by design ⁹) will cause those commands to fail. The research suggests multi-stage builds as a solution (use a `-dev` image with a shell for build steps, and a distroless for runtime) ¹⁰. Our tool might not automatically refactor into multi-stage (complex), but it can detect this scenario (e.g. a `RUN apt-get install` in the Dockerfile) and warn the user that manual intervention is needed to adopt a distroless image.
- The **entrypoint or command** might need adjustment. For instance, if the Dockerfile ends with `CMD ["/start.sh"]` and expects a shell to interpret it, that might not work on a non-shell base. Ideally, entrypoints in distroless images point directly to the binary (no shell) ⁹. The tool can't universally fix this, but could at least flag it.
- **User/Permission** differences: Chainguard images often run as non-root by default ²⁰. If the Dockerfile or application expects root or certain UIDs, there could be permission issues. Our tool might mention this in documentation or as a post-PR note (e.g. "Chainguard images run as nonroot user by default ²¹; ensure your application is compatible or adjust file permissions as needed.").
- **Performance:** Scanning images with Trivy will involve downloading vulnerability databases (Trivy does this on first run) and possibly pulling container images. This can take a few minutes for large images. Caching could be considered (Trivy caches scan results), but for a single-run CLI it may not matter much. We should at least allow the user to skip scans if speed is a concern (e.g. a `--no-scan` mode just to do the rewrite).
- **Output Volume:** Trivy outputs can be very verbose (listing every vulnerability). We might default to a summary in logs, and save full JSON reports for those who want detail. This balances clarity with thoroughness.

Despite these challenges, the value of the tool is clear: it automates what is otherwise a tedious, error-prone process of identifying a safer base image, making the substitution, and verifying improvement. By leveraging **Trivy's vulnerability scan and SBOM generation** features (which can be invoked with simple CLI flags ²² ¹⁴), the tool provides tangible evidence of security gains. Combined with automated PR creation, it integrates into DevOps workflows, allowing teams to review and merge security upgrades easily.

Prior Art and Inspiration

This concept touches on areas of existing tools: - **Trivy GitHub Actions/CI:** Trivy is commonly used in CI pipelines to fail builds if vulnerabilities exceed a threshold ²³. However, those typically report issues but don't fix them. Our tool goes further to propose a fix (upgrading base image) and automates it. - **Dependabot & Renovate:** These tools update dependencies automatically and can also bump Docker base image tags. For example, RenovateBot can submit PRs to update to a newer version of a base image tag ²⁴. Our tool differs by suggesting *different* base images (e.g. Debian to Chainguard) for security, not just newer versions. - **Salesforce's Dockerfile Image Update:** An open-source tool by Salesforce can scan GitHub for Dockerfiles and update them to a desired base image/tag, then open PRs ²⁵ ²⁶. It focuses on keeping child images in sync with parent base updates. Our tool is narrower (operating likely on a given repository rather than searching org-wide) but similarly automates PRs. It confirms the feasibility of the approach: scanning for `FROM` lines and programmatically creating PRs is a proven concept ²⁷. - **Chainguard's Dockerfile recommendations:** Chainguard provides education on using their images, often suggesting multi-stage builds ²⁸. While not a tool, it indicates the expected pattern (builder image with dev tools, then minimal runtime image). - **Manual Guides:** Numerous blogs recommend switching to distroless/Chainguard for security, documenting reduced CVE counts ⁵. This tool essentially automates those best practices.

In summary, the research underlines that a Go-based CLI tool can effectively combine: - Dockerfile parsing & modification, - Trivy scanning & SBOM output, - and GitHub automation, to significantly harden container images. By using **Chainguard's Wolfi** or **Google's Distroless** as drop-in bases, organizations can remove the “low-hanging fruit” vulnerabilities in their containers (those coming from the OS layer) ⁶. This complements other security efforts (like fixing application dependency vulns) and fits into a modern DevSecOps pipeline where **security improvements are delivered as code changes (via PR)** for easy review and integration.

Plan

Functional Flow

The following outlines how the tool will function from a user's perspective and the flow of operations internally:

1. **CLI Invocation:** The user runs the CLI, pointing it to a Dockerfile or a directory (repository). For example:

```
$ secureDockerfile scan --path ./my-app --base=chainguard --github-pr
```

- Flags might include:

- `--path` or positional argument for the target Dockerfile/repo.
- `--base` to specify preference (`chainguard` or `distroless`); default could attempt Chainguard first, then fallback to Distroless if not available).
- `--github-pr` (optional) to enable PR creation (assumes `GITHUB_TOKEN` env is set or `--token` provided).
- `--output-dir` for reports (default `./secure-dockerfile-report`).
- `--skip-sbom` or `--skip-scan` if user wants to only rewrite without scanning.

- **Discovery of Dockerfiles:** The tool scans the provided path for Dockerfile(s). By default, it will look for files named “Dockerfile” or with `.Dockerfile` extension in the directory (recursively, if multiple services). It will exclude certain paths like `node_modules` or others if scanning whole repo.

- **Dockerfile Parsing:** For each Dockerfile found, parse it to identify:

- Base images in `FROM` lines. In multi-stage, note the final stage's base separately.
- Whether the Dockerfile uses a single stage with package installation (which might indicate a need for multi-stage conversion).
- Current base image names and tags.

- **Secure Base Lookup:** For each identified base image, determine the replacement:

- If `--base=chainguard`, use Chainguard's image if available. E.g., `golang:1.20-alpine` → `cgr.dev/chainguard/go:latest` (Chainguard's Go image, which is Wolfi-based).

- If `--base=distroless`, use Google Distroless equivalent. E.g., `python:3.11-slim` → `gcr.io/distroless/python3:latest`.
- The tool may have a built-in map for common images. If not found, and if the base is Alpine, it might default to `cgr.dev/chainguard/static:latest` (a Wolfi base with just BusyBox and APK).
- In cases where no direct mapping is known, the tool will log “No secure base available for X – skipping this Dockerfile” or similar.
- **Note:** In future, the tool could query an API or a local database of image CVE counts to pick the truly most secure base. For now, Chainguard and Distroless are generally safe bets (Chainguard even provides a **zero-CVE guarantee at build time** ²).
- **Trivy Scan – Original Base:** If scanning is enabled, for each base image (or each Dockerfile):
 - Pull the image (using Docker daemon or `crane` library to fetch the image layers if we want to avoid Docker).
 - Run `trivy image` on it to get vulnerability info. Possibly use `--format json` to parse the output in code, or `--format table` to just log it.
 - Alternatively, run `trivy image --format cyclonedx -o original-sbom.json <image>` to get the SBOM then `trivy sbom --scanners vuln original-sbom.json` to get vuln count from SBOM ¹⁷ ¹⁸. But a direct image scan is simpler.
 - Save the summary (total vulns by severity) for reporting.
- **Dockerfile Rewrite:** Replace the base image lines with the secure alternatives determined in step 4. The AST is modified and the Dockerfile is written out (either in place or to a new file if we want to preserve original—likely in place, but we could backup original as `.bak`).
- If a multi-stage conversion is needed (detected in step 3), this is tricky to automate. MVP might just warn. E.g., “Consider refactoring this Dockerfile to multi-stage: base image has no shell, but RUN commands exist that use shell.”
- Ensure formatting is preserved reasonably (the parser will lose comments or order if not careful, but BuildKit’s parser should maintain structure/comments in the AST so we can output similarly).
- **Trivy Scan – New Base:** After rewriting, if possible build the new image (if context is available and user has Docker). This step might be **optional** due to the complexity of building user’s app. Instead, we likely just scan the new base image:
 - Run `trivy image` on the new base (e.g. `cgr.dev/chainguard/python:latest`).
 - If we did build the full image, scan that.
 - Again, collect vulnerability summary and generate SBOM (`new-sbom.json`).
- **Comparison & Logging:** Now the tool compares the before and after:
 - Log an **informational report**: e.g.

```
Dockerfile ./my-app/Dockerfile:
- Original base: python:3.11-slim (Size: 200 MB, Vulns: 30, High: 5,
Critical: 1)
- New base: cgr.dev/chainguard/python:latest (Size: 85 MB, Vulns: 0,
High: 0, Critical: 0)
=> Replaced base image, reduced OS package CVEs by 100% 2.
```

The reference to reduction is backed by Chainguard's zero-CVE stance.

- If multiple Dockerfiles, do this for each, and perhaps a summary total (e.g. "Total critical vulns eliminated: X across Y images").
- Save the logs (the console output can be redirected or we might allow `--log-file`).
- Possibly output a markdown or HTML report file combining these results for easy viewing (stretch goal).
- **GitHub PR (if enabled):** The tool will:
 - Check that the directory is a git repo (or the user provided a GitHub repo link).
 - If not a git repository already, the tool may clone the repo (if `--repo-url` was given).
 - Create a new branch, commit the changed Dockerfiles and added report files (SBOMs, etc. could be attached or posted to an Issue instead of committing binary JSON files—perhaps better to avoid committing SBOM JSONs to the code repo, and instead attach them in the PR comment).
 - Use the GitHub API to open a PR from that branch to the main branch. The PR body can include the before/after summary, and maybe links to full log or SBOM gist.
 - If multiple Dockerfiles were changed, it could be one PR with all changes (preferred).
 - The tool prints the PR URL or number upon success.
- If any step fails (e.g. auth issue, or user has no push access), log a warning. The user can then manually create a PR using the committed branch.
- **Completion:** Exit with a code (0 if everything succeeded and no critical vulns remain, perhaps non-zero if some Dockerfiles couldn't be processed or vulnerabilities remain above a threshold – this could be configurable to fail CI if still too many vulns).

Throughout this flow, **error handling** is important (e.g. network issues pulling images, Trivy not installed, etc.). The tool should fail gracefully with clear messages.

Example CLI Usage and Output

Suppose we have a simple Dockerfile:

```
# ./app/Dockerfile (original)
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN apk add --no-cache git
```

```
COPY . .
RUN npm install && npm run build
CMD ["node", "dist/server.js"]
```

If we run `secureDockerfile scan --path ./app --base=chainguard`, the tool might produce:

• **Console Output (excerpt):**

```
Scanning ./app/Dockerfile...
Found base image: node:18-alpine
Scanning vulnerabilities in node:18-alpine...
... [Trivy output] ...
Recommendation: Use cgr.dev/chainguard/node:latest (Wolfi base) as a drop-in
replacement for Node.js 18.
Scanning vulnerabilities in cgr.dev/chainguard/node:latest...
... [Trivy output] ...
Original base 'node:18-alpine': 12 vulnerabilities (High: 3, Critical: 1)
New base 'cgr.dev/chainguard/node:latest': 0 vulnerabilities (High: 0, Critical:
0)
Rewriting Dockerfile...
Dockerfile ./app/Dockerfile updated.

Summary for ./app/Dockerfile:
  Base Image before/after: node:18-alpine → cgr.dev/chainguard/node:latest
  Critical/High vulns before/after: 1 Critical, 3 High → 0 Critical, 0 High 5
  Total image size before/after: ~150 MB → ~50 MB (estimated)
  Note: Removed the use of 'apk' - the new base has no package manager 9.
```

• **Rewritten Dockerfile:**

```
FROM cgr.dev/chainguard/node:latest
WORKDIR /app
COPY package*.json ./
# Git is not available in distroless; consider installing in a build stage if
needed.
# RUN apk add --no-cache git <-- removed or moved to builder stage
COPY . .
RUN npm install && npm run build
CMD ["node", "dist/server.js"]
```

(Comments have been added by the tool to guide the user about removed/modified lines.)

• **Generated Files:**

- `app_original.sbom.json` and `app_new.sbom.json` – CycloneDX SBOMs for the original and new images.

- `app_trivy_original.txt` and `app_trivy_new.txt` – full vulnerability scan reports from Trivy for detailed review.
- These files might be referenced in the PR or saved in the specified output directory.

If `--github-pr` was provided and configured, a PR would be created on GitHub, which the user could then review and merge.

Project Directory Structure (Proposed)

Organizing the project well will aid development and maintenance:

```
secure-dockerfile/
├── cmd/
│   └── securedockerfile/      # main package for CLI
│       └── main.go           # CLI argument parsing and command dispatch
├── pkg/
│   ├── dockerfile/
│   │   ├── parser.go         # functions to parse and modify Dockerfiles
│   │   │   (wraps moby parser)
│   │   └── rewrite.go        # logic to replace base images, handle multi-
stage
│   └── trivy/
│       ├── scanner.go        # functions to call Trivy (image scan, generate
SBOM)
│       └── types.go           # types for parsed scan results (if JSON) or
summary struct
│   ├── recommender/
│   │   └── base_lookup.go     # mapping from base image -> secure alternative
│   ├── github/
│   │   └── pr.go              # functions to create PR (using go-github or HTTP
calls)
│   └── logging/
│       └── logger.go          # setup for logging (could just use logrus
globally)
├── internal/                 # (if needed, for internal helpers not exposed as
API)
├── reports/                  # (runtime) default directory to output SBOMs/
reports
├── go.mod, go.sum
└── README.md
```

In the above structure: - **cmd/securedockerfile** contains the CLI entry point. This will use something like Cobra or the standard `flag` package for argument parsing. Cobra is useful for subcommands (we might have subcommands like `scan` and in future maybe `verify` or others). - **pkg/dockerfile** encapsulates Dockerfile parsing and rewriting. It can expose a function like `ProcessDockerfile(path string, baseChoice string) (ChangesReport, error)` that does parsing, replacement, and returns a

summary of changes (along with maybe the new content or path to new file). - **pkg/trivy** deals with invoking Trivy. It might shell out (`os/exec` to call the `trivy` binary) or possibly use Trivy as a Go library (Trivy's code could be imported, but it might be heavy; calling the binary is simpler initially). This package might handle ensuring the DB is updated (running `trivy --download-db` on first run to cache). - **pkg/recommender** holds the mapping logic. Perhaps a map or list of structs internally, or it could read from a JSON/yaml file of mappings to allow easy updates. - **pkg/github** uses the GitHub API. The `pr.go` might have `CreatePullRequest(repo, branch, files, token) error` which handles pushing a branch and creating PR. We might use a simpler route: since the user environment likely has git, we could call `git` CLI to push branch, then use API for PR. But using `go-git` or `exec.Command("git", ...)` are options. Using the GitHub API to push file contents directly is also possible (but more complex to commit multiple files). - **pkg/logging** simply configures global logging format (e.g. JSON or text with timestamps) and log level.

We will also include a **README.md** with usage instructions, and possibly example output (some of which we have drafted above).

Dockerfile Snippets – Before and After

It's helpful to see a concrete example of transformation. Consider a simpler case:

Before (in repository):

```
FROM python:3.9-slim-buster
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["python", "app.py"]
```

After (tool's output):

```
FROM cgr.dev/chainguard/python:latest    # Changed from python:3.9-slim-buster
WORKDIR /app
COPY . .
# Install dependencies in a builder stage or at build time, as runtime image has
# no shell or pip.
# For example, use cgr.dev/chainguard/python:latest-dev for building wheels,
# then copy into this image.
# RUN pip install -r requirements.txt <-- removed from runtime image
CMD ["python", "app.py"]
```

Here, the tool replaced the base image and commented out the `RUN pip install` with guidance, because the Chainguard runtime image doesn't include a package manager or even pip by default. The user (or an advanced future feature of the tool) would need to implement a multi-stage build: - Stage 1: `FROM`

cgr.dev/chainguard/python:latest-dev as builder, do pip install (with wheels output to a directory). - Stage 2: use cgr.dev/chainguard/python:latest and copy the installed libs.

While the MVP tool won't automate multi-stage conversion, highlighting this necessity is crucial for usability. This keeps the functional flow safe – we don't want to suggest a secure image and break the build without explanation.

Security and Compliance Features

It's worth noting how this plan aligns with security best practices: - By generating an **SBOM** for both versions of the image, we provide transparency. Users (and auditors) can see exactly what changed. Trivy's ability to itemize components means the SBOM will list, for example, that `glibc` or `OpenSSL` packages present in Debian are simply absent in the Wolfi image (or replaced by safer alternatives) ¹⁴. - Using Chainguard images means **provenance and signatures** are available upstream. Our tool can mention in logs that the new base is **attested and comes with an SBOM by default** ^{29 30}, which further enhances supply chain security. - We can integrate an optional step to verify Sigstore/cosign signatures of the new base image if we want to be thorough (Chainguard signs their images). - If policy requires, the tool could run `trivy config` on the Dockerfile as well to catch misconfigurations (Trivy can scan Dockerfile for bad practices via Rego policies). This is out-of-scope for now but could be an additional feature: e.g., warn if using `latest` tags (though ironically we might use `:latest` of Chainguard which is rolling but in their case kept updated continuously).

Future Extensions

While focusing on the core functionality, we note some future improvements that emerged from research: - **Multi-Stage Auto-Refactor**: automatically split Dockerfile into builder and runtime stages if needed (e.g., detect package install commands and convert them to a builder stage using a `-dev` image). This is complex but would be a killer feature to fully automate transition to distroless images ¹⁰. - **Integration with CI/CD**: Provide a GitHub Action or CLI usage example that can be run in CI to automatically open PRs when a new secure base version is available, similar to dependabot but for switching to Chainguard periodically or when CVE count threshold exceeded. - **Interactive Mode**: Possibly prompt the user if multiple options are available for a base image (e.g., multiple distroless variants). - **Metrics**: The tool could output metrics like “% CVE reduction” or even cost savings (Chainguard's site mentions hours saved in patching ³¹). - **Support other scanners or languages**: e.g., integrate Snyk or Grype in addition to Trivy for comparison, or extend to Docker Compose files (ensuring base images in services are updated).

The research confirms that implementing this tool in **Go** is appropriate: Trivy itself is written in Go (so integration is natural), and Go's ecosystem has libraries for Dockerfile parsing and GitHub API. Moreover, distributing a Go CLI is easy (single binary). Go's efficiency and concurrency could be useful if scanning multiple Dockerfiles in parallel.

By proceeding with the plan below, we aim to deliver a functional MVP in about 4 weeks, with a clear path for incremental improvements thereafter.

Implementation

Below is a detailed 4-week execution plan, broken into phases with tasks, roles, and time estimates. We assume a small team of **2-3 people** with Go and DevOps experience. The plan is designed for one **tech lead/senior developer** and one additional **developer**, with potential part-time input from a **DevOps engineer** (for CI setup or Docker environment help). Each week is a **phase** with specific focus and deliverables. The total project effort is roughly one month (4 weeks).

Week 1 – Project Setup & Design (Phase 1)

Goals: Establish the project structure, clarify requirements, and implement the foundation (parsing and basic rewriting).

Deliverables by end of Week 1: - Project repo initialized (Go module, basic README). - Dockerfile parsing and representation of base images. - A stub CLI that can list base images in a Dockerfile and suggest (but not apply) replacements. - Plan for Trivy integration (decide on CLI vs library usage).

Task	Role	Skills / Notes	Est. Hours
Requirements Refinement & Tech Design	Tech Lead	Go design, threat modeling for tool misuse	4 hours
Set up repository (Go module, linters, CI)	DevOps/ Developer	Initialize GitHub repo, CI pipeline for lint/test	4 hours
Research Go libraries (Dockerfile parse, Trivy)	Developer	Evaluate moby/buildkit parser, go-github, etc.	6 hours
Implement Dockerfile parsing module	Developer	Use BuildKit parser to extract FROM instructions 7 .	8 hours
Implement base image recommender (stub)	Developer	Hard-code a few mappings (e.g., ubuntu->wolfi)	6 hours
CLI Scaffolding (Cobra or flag parsing)	Developer	Create scan command, accept path & flags	4 hours
Test parsing on sample Dockerfiles	Developer	Unit tests with multi-stage, comments, etc.	4 hours
Review & Adjust (design review meeting)	Tech Lead + Dev	Ensure parsing covers edge cases, refine mappings	2 hours

Total Week 1 effort: ~34 hours (could be ~20h for Dev, ~8h for Lead, ~6h for DevOps).

By the end of Week 1, running `secureDockerfile scan --path /myrepo` would print detected base images and what it would replace them with (without actually rewriting yet). For example: "Detected base

image python:3.9-slim → would suggest cgr.dev/chainguard/python:latest.” This confirms our parser and mapping logic works on real files.

Week 2 – Core Development (Phase 2)

Goals: Implement the main functionality: rewriting Dockerfiles and integrating Trivy to generate reports. Begin handling edge cases (multi-stage, no direct mapping, etc.).

Deliverables by end of Week 2: - Dockerfile rewriting applied and output to file. - Trivy scanning integrated to get vulnerability data. - SBOM generation for before/after images. - Logging of results in a human-readable summary.

Task	Role	Skills / Notes	Est. Hours
Implement Dockerfile rewrite function	Developer	Modify AST and write file (preserve format as much as possible)	6 hours
Integrate Trivy scanning (via CLI calls)	Developer	Use <code>os/exec</code> to run <code>trivy image</code> , parse JSON output	8 hours
Implement SBOM generation with Trivy	Developer	<code>trivy image --format cyclonedx</code> to file ¹⁵	4 hours
Parse Trivy JSON output to summary struct	Developer	Simplify to counts of vulns by severity	6 hours
Develop logging/reporting module	Developer	Format comparison output, use logrus for structured logging	4 hours
Edge case handling in recommender	Developer	e.g., if base is already distroless, log skip; if none found	4 hours
Multi-stage awareness	Tech Lead	Ensure only final stage base is replaced unless specified	3 hours
Remove or comment incompatible instructions	Developer	Simple heuristic: comment out <code>apt-get</code> or <code>apk</code> if new base has no package manager, add note ⁹	5 hours
Testing scanning on a known vulnerable image	Developer	E.g., use <code>node:14</code> with vulns, ensure Trivy output is captured	3 hours
Midpoint Review (demo current state)	Tech Lead + Team	Run tool on sample repo, collect feedback on output format	2 hours

Total Week 2 effort: ~41 hours.

By end of Week 2, the tool should be capable of taking a Dockerfile, rewriting it, and outputting a basic report. For instance, running on a test file yields an updated Dockerfile and console output showing

vulnerability reduction. We will likely test on a small vulnerable image (maybe an outdated Node image) to see real numbers. We ensure the **SBOM files** are generated and saved.

The team will review output readability (is the summary clear to a developer? Do we need to add color, or maybe output as Markdown for the PR?). Minor adjustments to format can be made. Also, at this stage, update the README with usage instructions and an example.

Week 3 – GitHub Integration & Refinement (Phase 3)

Goals: Add the GitHub PR automation and polish the tool (fix bugs, improve user experience, add configuration options). Also start writing documentation and tests.

Deliverables by end of Week 3: - GitHub PR creation working (tested on a sample repository). - Configurable options (via flags or config file). - More robust error handling and user messages. - Increased test coverage for core logic.

Task	Role	Skills / Notes	Est. Hours
Implement GitHub auth & branch push	Developer	Use go-github or HTTP calls; create branch & commit	8 hours
Implement PR creation logic	Developer	POST /repos/:owner/:repo/pulls via API ³²	6 hours
Test PR flow on personal repo (sandbox)	Developer	End-to-end test: local changes -> PR on GitHub	4 hours
Error handling & messaging improvements	Developer	e.g., if Trivy not installed, if Dockerfile parse fails	5 hours
Add CLI flags for output paths, thresholds	Developer	E.g., <code>--fail-on-high</code> to exit non-zero if high vulns remain	3 hours
Logging enhancements (levels, verbosity)	Developer	e.g., <code>--verbose</code> flag for debug logs	2 hours
Write unit tests for recommender logic	Developer	Ensure mapping returns expected results	3 hours
Write unit tests for parsing edge cases	Developer	Dockerfiles with comments, multiple FROM, etc.	3 hours
Integration test with a real Trivy call	Developer	(Optional) use a lightweight image in CI to test scanning	4 hours
Documentation: usage & examples	Tech Lead	Expand README (installation, examples, limitations)	5 hours
Team Review (QA)	Tech Lead + Dev	Code review, ensure code quality and security	3 hours

Total Week 3 effort: ~43 hours. (Potentially second developer helps in parallel to cover testing & docs.)

By the end of Week 3, the project should be nearly feature-complete. We will demonstrate the tool on a real use-case: e.g., take an open-source project's Dockerfile that uses Ubuntu, run our tool, and show the PR on GitHub that updates it to Chainguard, with a nice summary of improvements. This would serve as a final verification and also marketing demo perhaps.

We'll also address any feedback from the Week 2 midpoint review: maybe formatting tweaks or additional warnings as needed.

Week 4 – Testing, Hardening, and Project Wrap-Up (Phase 4)

Goals: In the final week, focus on quality: thorough testing (including user acceptance tests), performance checks, and preparing for release (version 0.1.0). Also, evaluate if MVP goals are met and document any known issues or future work.

Deliverables by end of Week 4: - A tagged release (v0.1) of the tool. - A short demo presentation or video (optional, if for internal stakeholders). - Final documentation and possibly a blog post or internal report on the project outcome. - Plans for next steps (if any, like onboarding more repos or adding features).

Task	Role	Skills / Notes	Est. Hours
Extensive testing on diverse Dockerfiles	Developer(s)	Try multi-stage Dockerfile, unusual bases, etc.	6 hours
Fix bugs found in testing	Developer	Memory, error edge cases, concurrency if any	6 hours
Performance optimization (if needed)	Developer	e.g., skip re-scanning same base image twice	3 hours
Security review	Tech Lead	Ensure no hardcoded secrets, safe temp file handling	2 hours
Package distribution prep	DevOps/ Developer	Create a Docker image of tool or brew formula if needed	4 hours
CI/CD pipeline finalization	DevOps	Ensure tests run in CI, maybe nightly cron for DB update	3 hours
Write user guide or blog (optional)	Tech Lead	Explain how to use the tool and results achieved	4 hours
Team retrospective & next steps	All	Discuss what went well, what to improve	2 hours
Buffer time for unexpected issues	All	(Unforeseen bug fixes, adjustments)	4 hours

Total Week 4 effort: ~34 hours.

During testing, we might intentionally run on some known projects or official images: - e.g., NGINX Dockerfile (which uses debian) -> see how we handle that. - Multi-stage example like a Golang app Dockerfile (which might already be multi-stage). - Ensure that if the base is already Chainguard/Distroless, the tool either skips or informs appropriately (no change needed).

We will also verify the GitHub PR on a fork scenario if possible (maybe simulate that the user is not owner, how does our tool behave – likely we won't implement forking due to time, but document that user needs push access).

Finally, the **MVP goals** are essentially met if: - The tool can scan a Dockerfile, replace its base image to a secure image, and show the reduction in vulnerabilities. - Optionally create a PR with those changes. - It should handle at least the common patterns and not break the resulting Dockerfile (except where noted, and then it should warn clearly).

We expect at the end of 4 weeks to have an **MVP** CLI that one can run on a repository and get a PR that significantly improves base image security. The outcome would be that for sample projects, we indeed see those “X vulnerabilities eliminated” in the PR, confirming a tangible security improvement.

Team & Resource Considerations

- **Team Size:** 2 full-time engineers are sufficient for this 4-week project. A third person (perhaps part-time) can act as a reviewer and help with DevOps (CI, releases). Since a lot of tasks can be done sequentially, a small focused team is ideal. Communication overhead is minimal and everyone stays on the same page. If only 1 developer is available, the timeline might extend to ~6 weeks.
- **Skills Required:** Go programming is a must (for all developers). Familiarity with Docker and Dockerfile syntax is important to understand edge cases. Security know-how (CVEs, SBOM) is needed to make good decisions (the Tech Lead likely provides this context). Experience with GitHub APIs and CI tools will help in automation aspects. Each member should ideally have a blend of these skills, but as roles:
 - Tech Lead: Go expertise, security context (CVEs, distroless knowledge), architecture design, ensures that the solution is robust and addresses the problem.
 - Developer: Focus on implementation, writing tests, integrating tools like Trivy.
 - DevOps (if separate): Ensure Trivy is available (perhaps installed in CI runner or as Docker), handle any pipeline needed for automated testing and releasing the tool.
- **Time Allocation:** Each week's plan shows ~30-40 hours of work; the team should budget some extra time for code reviews and coordination. Stand-ups or syncs 3 times a week could help keep progress on track given the short timeline.
- **Risks and Mitigations:**
 - **Trivy dependency:** If Trivy changes or has issues (e.g., DB download failures), our tool might be blocked. Mitigation: pin a Trivy version and allow using a cached DB or offline mode if needed.
 - **Chainguard/Distroless availability:** If pulling those images is slow or they deprecate tags, we should handle errors. Maybe always use `:latest` for Chainguard (which they keep updated ⁵).
 - **False sense of security:** Replacing the base doesn't solve all issues (app dependencies might still be vulnerable). We will note in documentation that this tool addresses base image vulns primarily, and is one part of a holistic security strategy.

MVP Outcome and Future Work

MVP Success Criteria: By the end of the implementation, the team should achieve: - A user can run the tool on a repository with vulnerable base images and get a clear suggestion or automated change that uses a secure base. - The resulting container image (if built) should have significantly fewer or zero known vulnerabilities in its OS packages, as confirmed by Trivy scans (the tool's own output will show this). - A sample PR generated by the tool should be easily understandable and mergeable, with minimal manual fixes needed by maintainers. - The tool's usage should be documented, and it should be reasonably easy to install (perhaps `go install ...` or a binary release). - All of this done within ~4 weeks of work, meaning scope is controlled to essential features.

Outcomes: - Immediate outcome: A working CLI that can be used internally or by open-source projects to improve container security. This could directly reduce the CVE count across many images (imagine running it on dozens of Dockerfiles; it automates days of manual work and ensures consistency). - Long-term outcome: If adopted, this could feed into CI processes (e.g., run weekly to catch when base images could be updated) and potentially integrate with vulnerability management programs (some organizations might integrate it with Jira or security dashboards via the reports). - The project also produces artifacts (SBOMs, logs) that can be archived for compliance, showing due diligence in reducing vulnerabilities.

Next Steps (beyond MVP): After delivering the MVP, the team might consider adding: - Multi-stage conversion assistance (making it truly one-click to go distroless). - Support for other base image sources like Red Hat UBI minimal images for those who need licensed support. - A mode to automatically open PRs in multiple repositories (scanning an entire GitHub org) – essentially a bot mode. - Monitoring of upstream base image releases (subscribe to Chainguard updates) to trigger the tool.

But these are outside the initial 4-week scope. The priority is to **deliver a reliable, usable tool** that addresses the core problem: outdated, insecure base images, and to do so in a developer-friendly manner (with clear output and easy integration via PRs).

By structuring the implementation in well-defined phases and keeping the team small and focused, this plan maximizes the chances of success. We will achieve an MVP that demonstrates the value of using **Go**, **Trivy**, and **secure base images** together to enhance container security with minimal human effort.

1 5 9 10 21 Distroless Base Image Pitfalls. In the beginning dev could just throw... | by Martin Ng | Medium

<https://medium.com/@martin.soon.ng/distroless-base-image-pitfalls-e5a3a531f5c3>

2 20 29 30 Securing Software with Chainguard Zero-CVE Base Images against Advanced Persistent Threat Groups | by William Konitzer | Medium

<https://medium.com/@wkonitzer/securing-software-with-chainguard-zero-cve-base-images-against-advanced-persistent-threat-groups-300e26fbeb54>

3 19 31 Secure your container images with Chainguard

<https://www.chainguard.dev/containers>

4 6 13 Chainguard Users: Is Paying \$30K Per Docker Image Really Worth It? : r/cybersecurity

https://www.reddit.com/r/cybersecurity/comments/1ihy9sr/chainguard_users_is_paying_30k_per_docker_image/

7 parser package - github.com/moby/buildkit/frontend/dockerfile/parser - Go Packages

<https://pkg.go.dev/github.com/moby/buildkit/frontend/dockerfile/parser>

8 28 Getting Started with Distroless Container Images — Chainguard Academy

<https://edu.chainguard.dev/chainguard/chainguard-images/about/getting-started-distroless/>

11 12 14 15 16 22 Trivy Container Security Scanner & SBOM Tool | Vulnerability Detection Guide

<https://squareops.com/blog/trivy-the-ultimate-open-source-tool-for-container-vulnerability-scanning-and-sbom-generation/>

17 18 SBOM - Trivy

<https://trivy.dev/latest/docs/target/sbom/>

23 r/devops on Reddit: Docker image vulnerability scan: Are critical ...

https://www.reddit.com/r/devops/comments/1bdue7p/docker_image_vulnerability_scan_are_critical/

24 Automatically update a dockerfile when new versions of the base ...

<https://stackoverflow.com/questions/77423857/automatically-update-a-dockerfile-when-new-versions-of-the-base-image-are-availa>

25 26 27 GitHub - salesforce/dockerfile-image-update: A tool that helps you get security patches for Docker images into production as quickly as possible without breaking things

<https://github.com/salesforce/dockerfile-image-update>

32 open a pull request when a branch is pushed or updated - GitHub

<https://github.com/vsoch/pull-request-action>